Clinic Front-Desk Assistant Project Structure & Development Plan

Updated file ownership: the leader also owns RAG infrastructure, the LLM adapter, the health endpoint and integration tests.

Scope: Build the backend REST API only. The clinic already has a chat client; no website or frontend is required. Every patient message is handled safely through FastAPI and LangGraph, with PostgreSQL as the only database.

01 Recommended repository structure

Based on your folder screenshot. The three main application layers are divided by specific filenames on page 2. All remaining support modules are assigned to the team leader.

PATH	PURPOSE
`app/`	Python backend and AI workflows
`main.py`	Leader: FastAPI entry, API router and health endpoint
`api/`	Conversations, appointments and approvals endpoints
`graphs/`	states.py, routing.py and workflows.py
`services/`	Conversation, appointment and clinic-knowledge logic
`repositories/`	Leader: PostgreSQL persistence using parameterized SQL
`rag/`	Leader: retrieval infrastructure (Member 4: knowledge answers)
`llm.py`	Leader: real/fake LLM adapter and timeouts
`config.py`	Leader: validate environment variables
`db/`	Leader: SQL schema, seed data and initialization
`data/clinic_documents/`	Clinic-approved information only
`tests/`	Unit, API, database and integration tests
`experiments/`	Research spikes for R1, R2 and R3
`docs/`	Architecture, research notes and ADRs

Leader-owned root configuration: `Dockerfile`, `compose.yaml`, `.env.example`, `pyproject.toml`, README.md, AGENTS.md, CHANGELOG.md and `.github/workflows/ci.yml`.

02 Confirmed division of work

These assignments reflect the team's exact requested filenames. The leader owns infrastructure, repositories, RAG module, LLM adapter and health endpoint. Members 2-4 each own one API file, one graph file and one service file.

MEMBER	EXACT FILES / AREAS	MAIN RESPONSIBILITY
Member 1 TEAM LEADER	app/main.py app/repositories/ (repository SQL code) app/config.py; app/rag/ app/llm.py; health endpoint in main.py db/ (schema and seed) Dockerfile, compose.yaml .env.example; initial CI setup cross-module integration tests	FastAPI entry and health API; parameterized PostgreSQL repository queries; settings, database schema and seed; RAG retrieval support and fake/real LLM adapter; Docker/Compose and CI; integration tests, VM deployment, planning and coordination.
Member 2 CONVERSATIONS	app/api/conversations.py app/graphs/states.py app/services/conversations.py	Create and resume patient conversations; send messages and return responses; define graph state and conversation fields; manage bounded message history and coordinate checkpoint storage with the leader.
Member 3 APPOINTMENTS + ROUTING	app/api/appointments.py app/graphs/routing.py app/services/appointments.py	Expose slot search and booking endpoints; implement booking service and prevent double bookings with leader-owned SQL functions; classify and route patient intents, including urgent safety routing.
Member 4 APPROVALS + WORKFLOWS	app/api/approvals.py app/graphs/workflows.py app/services/clinic_knowledge.py	Expose pending approvals and approve/reject decisions; connect LangGraph workflow branches and real interrupt/resume; implement document-based clinic answers with source names and reliable fallback.

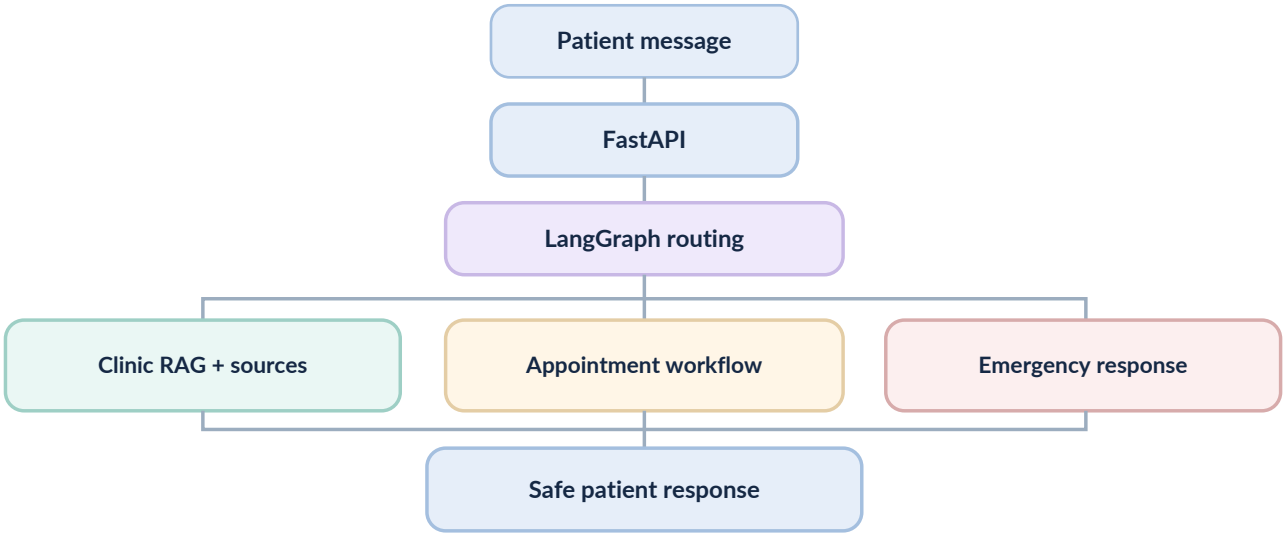
New leader assignments: `app/rag/`, `app/llm.py`, health endpoint (in main.py) and cross-module integration tests. Member 4 still owns `services/clinic_knowledge.py`; coordinate the retrieval interface before coding.

Issue #3 research (3 members; leader excluded)

OWNER	TOPIC	EXPECTED FINDING
Member 2	R1: Checkpoints	Conversation state stored in PostgreSQL survives restart
Member 3	R2: Human in the loop	Approval interrupt remains pending and resumes after staff decision
Member 4	R3: RAG	Replies cite clinic documents; missing evidence gives honest fallback

Coordination: Member 3 researches human approval, but Member 4 owns the approvals API and workflow implementation. They should share the research decisions before coding. The leader sets up the repository, PostgreSQL, Docker, initial CI and ClickUp during the research block.

03 How the system works



Flow: The patient sends a message to FastAPI. LangGraph checks for an emergency first, identifies the request and sends it to the appropriate workflow. Clinic questions use document retrieval; appointment requests check PostgreSQL. Cancellations and same-day bookings pause for front-desk approval. The API returns a safe response.

Essential shared contracts

- Conversation ID: A consistent identifier connects saved conversation records with the LangGraph thread ID.
- Workflow state: Agree on fields for recent messages, intent, requested slot and approval status.
- Data safety: Use PostgreSQL only, with parameterized SQL through psycopg; do not use an ORM.
- Booking protection: Enforce a database-backed atomic operation that rejects simultaneous double-bookings.
- Approval: Use a real LangGraph pause/resume; approval must survive a container restart.
- Model testing: A deterministic fake LLM drives CI; handle model/database errors and external timeouts.

Safety rule: The assistant must never diagnose or recommend treatment. An emergency must be answered immediately with emergency instructions and flagged for staff, without passing through ordinary booking or RAG flows.

04 Development timeline

PHASE / TIME	ACTION	DELIVERABLE
Day 1 / 09:00-09:20	Read requirements; choose leader	Shared understanding
Day 1 / 09:20-10:20	3 parallel research spikes; leader sets up repo, DB, Compose and CI	Experiments, notes, initial repo
Day 1 / 10:20-12:00	Research share-out, architecture and contracts	ADRs; diagrams, endpoints and data model
Day 1 / 12:00-13:00	Backlog, story points, owners and sign-off	Sprint ready; ClickUp updated
Day 1 / 14:00-18:00	Build block 1 and merge reviewed PRs	DB + API foundation; first vertical flows
Day 2 / 09:00-15:00	Build block 2, integrate, test	All Must requirements; fresh-clone proof
Day 2 / 15:00-16:30	Code freeze, release and deployment	Tag v0.1.0; VM smoke test
Day 2 / 16:30-18:00	Review, demo and retrospective	Team evidence + individual walkthroughs

05 First two code tasks by member

OWNER	FIRST APPLICATION-CODE TASK	SECOND APPLICATION-CODE TASK
Leader	Create main.py (including health) and config.py; build app/llm.py real/fake adapter	Build SQL repositories, DB schema, Docker/CI, RAG retrieval helpers and integration tests
Member 2	Build conversation endpoints in api/conversations.py and conversation service	Implement graphs/states.py and persistent/bounded conversation-state integration
Member 3	Implement appointments API and service with safe slot lookup and booking	Build graphs/routing.py with all required intents and emergency routing
Member 4	Implement approvals API and LangGraph interrupt/resume in graphs/workflows.py	Implement clinic_knowledge.py with document-grounded answers and source names

Integration: The leader defines repository contracts and SQL schema with Members 2-4. Member 2 publishes graph-state fields; Member 3 publishes routing decisions; Member 4 connects all branches in workflows.py. Nobody edits another member's assigned file without agreeing first.

06 Build and release quality checklist

- All Must features FR-1 to FR-7 work end-to-end on the Ubuntu Server VM.
- Leader-owned PostgreSQL schema and repositories store doctors, slots, bookings, approvals, conversations and checkpoints.
- Clinic answers use only approved documents and name their source; unknown answers are not invented.
- Every cancellation and same-day booking pauses for approval, survives a restart and resumes correctly.
- Leader and Member 3 verify atomic appointment booking: two simultaneous requests cannot reserve one slot.
- Each member adds tests for their own modules; CI passes Ruff lint/format, mypy and pytest at 80%+ coverage.
- Main is protected, needs two approvals, and code is merged through reviewed feature-branch PRs.
- No secrets are committed; required settings are in environment variables and .env.example.
- Fresh-clone setup uses a single docker compose command; database initializes automatically.
- Tag v0.1.0, smoke test the VM deployment, update release notes and demonstrate rollback.
- Research findings are recorded in docs/research/ and design choices in docs/adr/.
- Scrum planning, standup, scope decisions and retrospective are recorded in ClickUp.

07 Recommended next steps

STEP	ACTION
1	Finish GitHub Issue #1: read and understand the requirements.
2	Finish Issue #2: project planning and distribution, including architecture and backlog.
3	Complete Issue #3 research in parallel: R1, R2 and R3, excluding the team leader.
4	Agree on shared contracts, then create small implementation issues linked to ClickUp tickets.
5	Build, review, integrate, test and deploy the v0.1.0 release before the freeze.

Important: This plan uses app/graphs/states.py exactly as requested (the earlier structure showed state.py). Choose one filename consistently before coding. The leader also owns app/rag/, app/llm.py, health API and integration testing. This is a substantial load; size and distribute the actual story points so the leader does not exceed 30% and all members meet the two application-code PR minimum.

Reference: User-provided Clinic Front-Desk Assistant - Team Challenge requirements and proposed folder structure screenshot.